Unicorn Magic — Claude Code Handoff Brief

Goal: Ship a paid iOS app to the App Store (~$1.99–$2.99) built around an interactive AI-powered unicorn experience for children ages 4–8.

---

1. What Exists Today

A fully working HTML/JS prototype (`unicorn_magic_app.html`) that demonstrates all 5 features in an iOS-style phone frame. The prototype runs in any browser. Claude Code's job is to rebuild this as a production React Native (Expo) app ready for App Store submission.

The prototype has:

- A custom SVG unicorn illustration with 4 colorable zones (body, mane, horn, tail)
- 5 tabs, all working: Dress, Facts, Color, Stories, Potions
- Claude API integration for AI-generated content (facts, stories, potions)
- Drag-and-drop + tap color interaction
- A paint/coloring-page mode

The prototype file is the source of truth for UX flow, colors, and feature behavior. Treat it as the design spec.

---

2. Recommended Tech Stack

Layer	**Choice**	**Reason**
Framework	**Expo (React Native)**	Fastest path to App Store; great SVG support; no native Xcode-only code needed initially
Language	**TypeScript**	Safer for a distributed product
SVG	**react-native-svg**	Direct port of the existing SVG unicorn

Layer	Choice	Reason
Navigation	**Expo Router** (tab-based)	Built-in, matches the 5-tab layout
State	**Zustand**	Lightweight global state (replaces the `S` object in the prototype)
API calls	**Backend proxy (required — see §4)**	Never expose Anthropic API key in a mobile app binary
Backend	**Cloudflare Workers or Vercel Edge Function**	Cheap/free tier, low latency, easy deploy
Monetization	**expo-in-app-purchases**	For paid download or consumable packs
Analytics	**PostHog (optional)**	Privacy-friendly, COPPA-compatible

---

3. App Architecture

```
unicorn-magic/
├── app/
│   ├── (tabs)/
│   │   ├── dress.tsx       # Tab 1: Dress-up with drag/tap color zones
│   │   ├── facts.tsx       # Tab 2: AI-generated unicorn facts
│   │   ├── color.tsx       # Tab 3: Coloring page mode
│   │   ├── stories.tsx     # Tab 4: AI stories with mood selector
│   │   └── potions.tsx     # Tab 5: AI magic potion recipes
│   └── _layout.tsx         # Root layout with tab bar
```

```
├── components/
│   ├── UnicornSVG.tsx       # The unicorn illustration (see §5)
│   ├── ColorSwatch.tsx
│   └── ZonePill.tsx
├── store/
│   └── unicornStore.ts      # Zustand state (colors, paint, mood, etc.)
├── api/
│   └── claude.ts            # Calls YOUR backend proxy, not Anthropic directly
├── backend/                 # Separate deploy (Cloudflare Worker or Vercel)
│   └── proxy.ts             # Holds ANTHROPIC_API_KEY in env, forwards requests
└── assets/
    └── (sounds, fonts, icon)
```

---

4. Critical: API Key Security

The prototype calls Anthropic directly from the client. This CANNOT ship to the App Store.

Anyone who decompiles the app binary would find the API key. You need a thin backend proxy.

Cloudflare Worker proxy (copy-paste ready):

```
// backend/worker.ts — deploy to Cloudflare Workers

export default {
  async fetch(request: Request, env: Env): Promise<Response> {
    if (request.method !== 'POST') return new Response('Method not allowed', { status: 405 });
```

```
    const body = await request.json();

    const response = await fetch('https://api.anthropic.com/v1/messages', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'x-api-key': env.ANTHROPIC_API_KEY,
        'anthropic-version': '2023-06-01',
      },
      body: JSON.stringify({
        model: 'claude-haiku-4-5-20251001', // Use Haiku — cheaper, fast enough for this
        max_tokens: 400,
        messages: body.messages,
        system: body.system,
      }),
    });
    const data = await response.json();
    return new Response(JSON.stringify(data), {
      headers: { 'Content-Type': 'application/json', 'Access-Control-Allow-Origin': '*' },
    });
  },
};
```

```
interface Env { ANTHROPIC_API_KEY: string; }
```

Why Haiku instead of Sonnet? Haiku is ~10x cheaper and responds in ~1s. For 2-sentence facts and short children's stories, it's more than capable. Your cost per user interaction drops to fractions of a cent.

App-side API client:

```
// api/claude.ts

const PROXY_URL = 'https://your-worker.your-subdomain.workers.dev';

export async function askClaude(system: string, prompt: string): Promise<string> {
  const res = await fetch(PROXY_URL, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      system,
      messages: [{ role: 'user', content: prompt }],
    }),
  });
  const data = await res.json();
  return data.content[0].text;
}
```

---

5. The Unicorn SVG

The SVG unicorn lives in the prototype file. When porting to `UnicornSVG.tsx`:

- Use `react-native-svg` — import `Svg, Path, Ellipse, Circle, Line, Rect, Text` from it
- The viewBox is `"0 0 312 272"`, rendered at `width={268} height={233}`
- **4 colorable zones:** `body`, `mane`, `horn`, `tail` — each zone's fill color comes from props
- In **Dress mode**: zones respond to `onPress` and receive dropped colors
- In **Color/paint mode**: same zones but filled white initially, tap to flood-fill with selected color
- Sparkle characters render conditionally based on a `sparkles` boolean prop

Component signature to implement:

```
interface UnicornSVGProps {

  colors: { body: string; mane: string; horn: string; tail: string };

  mode: 'dress' | 'paint';

  sparkles?: boolean;

  onZonePress: (zone: 'body' | 'mane' | 'horn' | 'tail') => void;

}
```

---

6. Design Tokens

```
export const COLORS = {

  // App chrome

  deepPurple: '#26215C',

  midPurple: '#534AB7',

  softPurple: '#3C3489',

  lavenderBg: '#EEEDFE',

  lavenderBorder: '#CECBF6',

  lavenderMuted: '#AFA9EC',
```

```
  textDark: '#444441',
  textMuted: '#888780',
  textDisabled: '#B4B2A9',
  // Unicorn palette (12 colors)
  palette: [
    '#ff9de2', '#c084fc', '#93c5fd', '#86efac',
    '#fda07a', '#fde68a', '#5eead4', '#fb7185',
    '#ddd6fe', '#ffd700', '#a5f3fc', '#f8f0ff',
  ],
  // Default unicorn colors
  defaultUnicorn: { body: '#f0e4ff', mane: '#ff9de2', horn: '#ffd700', tail: '#c084fc' },
};
export const FONT = {
  regular: 400,
  medium: 500,
};
```

---

7. Feature Specs (Tab by Tab)

Tab 1 — Dress ◈

- Show unicorn SVG with current colors applied
- 4 zone pills below: Body, Mane, Horn, Tail — highlight selected zone
- 12-color swatch grid — tap swatch while zone selected → applies color

- On native: use long-press + drag on swatches to drag onto the unicorn zones (or simplify to tap-zone → tap-color flow, which is what most kids will do anyway)
- Toggle switch: Magic Sparkles (adds decorative star characters around unicorn)
- Reset button → restores default colors
- State persists across sessions via AsyncStorage

Tab 2 — Facts ✦

- Tap button → calls Claude → displays fact in purple card
- Previous facts stack below in white cards (newest on top)
- System prompt: `"You generate magical, age-appropriate unicorn facts for children aged 4-8. Always be whimsical, brief (2-3 sentences), and educational. No scary content."`
- User prompt: `"Give me one fun unicorn fact."`
- Fallback fact hardcoded for offline/error state

Tab 3 — Color ◑

- Same unicorn SVG but all zone colors start white
- Color palette at top — tap to select active paint color (ring indicator)
- Tap any unicorn zone → floods that zone with active color
- Clear button → resets all zones to white
- Consider saving coloring progress to AsyncStorage per session

Tab 4 — Stories ≡

- 5 mood chips: magical adventure / friendship / silly and funny / brave hero / mystery
- Tap "Tell me a story!" → Claude generates JSON `{title: string, text: string}`
- System prompt: `"You write short, whimsical unicorn stories for children aged 4-8. Always age-appropriate, positive, and enchanting. Return only valid JSON with 'title' and 'text' fields. No markdown."`
- User prompt: `"Write a short [mood] unicorn story. 3-4 sentences."`
- Display: dark purple header with title, white body with text
- Consider a "Read aloud" button using `expo-speech` — great accessibility feature for young kids who can't read yet

Tab 5 — Potions ◆

- Tap "Brew a magic potion!" → Claude generates JSON with `name`, `effect`, `ingredients[]`, `instructions`
- System prompt: `"You create whimsical, imaginative magic potion recipes for children aged 4-8. Always positive, silly, and safe.`

`Ingredients should be fantastical (moonbeam flakes, dragon giggles). Return only valid JSON. No markdown."`

- Display potion card with ingredient bullets (purple dots) and instructions section
- "Brew a different potion" → clears and re-prompts

---

8. App Store Requirements Checklist

Before submission:

- ☐ **Apple Developer Account** — $99/year at developer.apple.com — Mark needs this
- ☐ **App icon** — 1024×1024px PNG, no transparency, no rounded corners (Apple applies mask)
- ☐ **Privacy Policy URL** — Required. Host a simple page. The app collects no personal data, making this easy. Template: "We do not collect or store any personal information. AI content is generated in real-time and not stored."
- ☐ **Age Rating** — Rate as **4+** (no objectionable content). This gets it featured in Kids category.
- ☐ **Kids Category** — If you list in Kids, Apple restricts third-party analytics and ads entirely. That's fine for this app. Do list here — it's where the parents look.
- ☐ **COPPA compliance** — Since the app targets children under 13, no behavioral tracking, no accounts, no data collection. The proxy backend must not log user content.
- ☐ **App Privacy labels** — Declare "Data Not Collected" in App Store Connect. This is a selling point — parents love seeing this.
- ☐ **Screenshots** — Need iPhone 6.9" (iPhone 16 Pro Max) and optionally iPad. Show each of the 5 tabs.
- ☐ **App Preview video** — Optional but increases conversion rate significantly. 15–30 seconds.

Monetization options:

Model	**Pros**	**Cons**
Paid upfront ($2.99)	Simple, no IAP code, parents prefer it	Lower download volume
Free + IAP unlock ($1.99)	More downloads, try before buy	More App Store review scrutiny
Free + Subscription ($0.99/mo)	Recurring revenue	Overkill for this app's scope

Recommendation: Paid upfront at $2.99. Parents of young children happily pay a few dollars for a quality, ad-free, safe app. The Kids category has strong conversion for paid apps. No subscriptions needed — the app content is AI-generated so it never goes stale.

Backend cost estimate at $2.99 price point:

- Haiku API cost: ~$0.001 per request (very rough)
- A heavy user might make 50 requests/month
- At 1,000 paid users: ~$50/month in API costs
- At 10,000 paid users: ~$500/month
- Cloudflare Workers free tier: 100,000 requests/day
- **Comfortable unit economics until you have thousands of DAU**

---

9. Nice-to-Have Features for V2

- **Read Aloud** — `expo-speech` reads stories and facts aloud. Huge for 4-6 year olds who can't read.
- **Save my unicorn** — `expo-media-library` to save a screenshot of the dressed unicorn to Camera Roll.
- **Unicorn name generator** — Claude generates a magical name for your unicorn. Easy, delightful.
- **Sound effects** — Subtle sparkle/magic sounds on interactions using `expo-av`.
- **Daily fact** — Push notification once/day with a unicorn fact (requires `expo-notifications` + backend scheduler).
- **More story moods** — Bedtime story, spooky (gentle), holiday themes.
- **Coloring page share** — Export the colored unicorn as an image.

---

10. Exact Prompt to Start Claude Code

Copy and paste this to kick off a Claude Code session:

I'm building a paid iOS app called "Unicorn Magic" for children ages 4-8 to sell on the App Store.

I have a complete working HTML/JS prototype that defines all features, UX flow, colors, and layout.

I also have a detailed handoff document. I need you to build this as a production Expo (React Native)

app in TypeScript, ready for App Store submission.

Please start by:

1. Initializing an Expo project: `npx create-expo-app unicorn-magic --template tabs`

2. Installing dependencies: react-native-svg, zustand, expo-router

3. Setting up the project structure as defined in the handoff doc

4. Porting the UnicornSVG component from HTML/SVG to react-native-svg

5. Implementing the Zustand store with the unicorn color state

6. Building the 5-tab layout matching the prototype's visual design exactly

Here are the design tokens:

- Background: #EEEDFE (lavender)

- Primary: #534AB7 (mid purple)

- Dark: #26215C (deep purple)

- Accent dark: #3C3489

- Border: #CECBF6

- Muted text: #888780

- Font: System (-apple-system)

- Tab bar: white background, 0.5px top border #CECBF6

The unicorn SVG viewBox is "0 0 312 272". The 4 interactive zones are: body, mane, horn, tail.

Each zone needs an onPress handler and accepts a fill color as a prop.

For API calls, do NOT call Anthropic directly. Create a stub `api/claude.ts` that calls

`https://REPLACE_WITH_YOUR_WORKER_URL` — I will deploy the Cloudflare Worker proxy separately.

Reference file: unicorn_magic_app.html (I will paste relevant sections as we go)

Reference doc: UNICORN_MAGIC_HANDOFF.md (this document)

Start with the project scaffold and UnicornSVG component.

---

11. Files to Hand to Claude Code

1. **`unicorn_magic_app.html`** — The complete prototype (paste inline or share the file)
2. **`UNICORN_MAGIC_HANDOFF.md`** — This document
3. Nothing else needed to start

---

12. Quick Reference: Current State Object

The prototype's global `S` object maps to your Zustand store:

```
// store/unicornStore.ts

interface UnicornStore {

  // Dress tab

  dressColors: { body: string; mane: string; horn: string; tail: string };

  sparkles: boolean;

  selectedZone: Zone | null;

  // Color tab

  paintColors: { body: string; mane: string; horn: string; tail: string };

  activePaintColor: string;

  // Stories tab
```

```
  storyMood: 'magical adventure' | 'friendship' | 'silly and funny' | 'brave hero' | 'mystery';
  currentStory: { title: string; text: string } | null;
  // Facts tab
  facts: string[];
  // Potions tab
  currentPotion: { name: string; effect: string; ingredients: string[]; instructions: string } | null;
  // Actions
  setDressColor: (zone: Zone, color: string) => void;
  toggleSparkles: () => void;
  setSelectedZone: (zone: Zone | null) => void;
  setPaintColor: (zone: Zone, color: string) => void;
  setActivePaintColor: (color: string) => void;
  resetDressColors: () => void;
  resetPaintColors: () => void;
  setStoryMood: (mood: string) => void;
  setStory: (story: { title: string; text: string } | null) => void;
  addFact: (fact: string) => void;
  setPotion: (potion: Potion | null) => void;
}
```

Document prepared April 2026 — Schectman Media Group LLC